République Algérienne Démocratique et
Populaire Ministère de l'Enseignement
Supérieur et de la Recherche Scientifique

Université des Sciences et de la Technologie Houari Boumediene

Faculté d'Informatique Département
des Systèmes Informatique
Spécialité :

Master 1 Ingénierie du logiciel

Multimedia Mini Project Assignment Simplified MPEG-4 Video Encoder Pipeline

Présenté par :

NOM : CHABANE	**NOM :** BIDA
PRENOM : KHAWLA	**PRENOM :** MAROUA
MATRICULE: 222236125020	**MATRICULE:** 212131034769

Introduction

This report explains the complete workflow implemented in the code. The goal is to build a simplified image and video compression system inspired by common transform and predictive coding principles. The system first prepares frames in a color space suitable for compression, then compresses intra-coded frames independently, and finally compresses predicted frames by storing motion information and residual errors.

The implementation is educational rather than a complete industrial codec. It demonstrates the main building blocks: YCbCr conversion, chroma subsampling, block DCT, quantization, I-frame coding, P-frame motion estimation, residual coding, and reconstruction.

1. Project Objective and Code Architecture

The project is divided into three Python modules. Each module corresponds to a stage of the compression process. This separation makes the pipeline easier to understand, test and explain.

File	Main role	Important functions
preprocessing (1).py	Prepare a BGR frame before compression by converting it to YCbCr and reducing chroma resolution.	bgr_to_ycbcr, ycbcr_to_bgr, subsample_420, preprocess_frame
intra_coding (1).py	Compress and reconstruct I-frames independently using 8x8 DCT and JPEG-like quantization.	scale_quantisation_matrix, dct2, idct2, encode_block, decode_block, encode_channel, decode_channel, encode_iframe, decode_iframe
inter_coding (1).py	Compress P-frames using a previous reconstructed frame, full-search motion estimation and residual coding.	full_search_block_matching, apply_motion_vectors, encode_residual_channel, decode_residual_channel, encode_pframe, decode_pframe

Global idea

An I-frame is compressed without using other frames. A P-frame is compressed by predicting it from a previous reconstructed frame, then encoding only the prediction error plus motion vectors.

Simplified Image Compression Pipeline

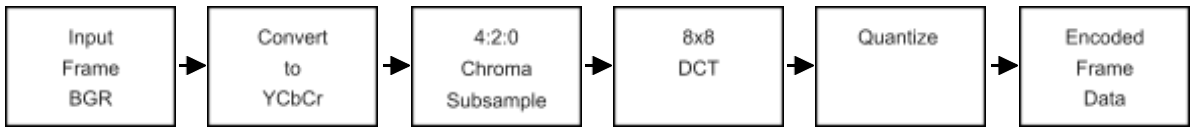


Figure 1 - Main compression flow used by the project.

PART 1: Preprocessing

Pre-processing prepares an input frame so that later compression stages can exploit visual perception. The input frame is read in BGR format, which is OpenCV's common channel order. It is converted to YCbCr, where Y contains brightness information and Cb/Cr contain color information.

1.1 Color Space Conversion: BGR to YCbCr

The function bgr_to_ycbcr(frame_bgr) receives a uint8 frame of shape H x W x 3. OpenCV converts the frame from BGR to YCrCb. Because OpenCV stores the channels as Y, Cr, Cb, the code manually reorders them to return Y, Cb, Cr. Each channel is converted to float32 for later mathematical operations.

```
def bgr_to_ycbcr(frame_bgr):
    ycrcb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2YCrCb).astype(np.float32)
    Y = ycrcb[:, :, 0]
    Cb = ycrcb[:, :, 2]
    Cr = ycrcb[:, :, 1]
    return Y, Cb, Cr
```

Channel	Meaning	Reason in compression
Y	Luminance: brightness and intensity.	Kept at full resolution because the human eye is sensitive to brightness details.
Cb	Blue-difference chroma component.	Can be stored at lower resolution with less visible quality loss.
Cr	Red-difference chroma component.	Can also be stored at lower resolution with less visible quality loss.

1.2 Complete Pre-processing Function

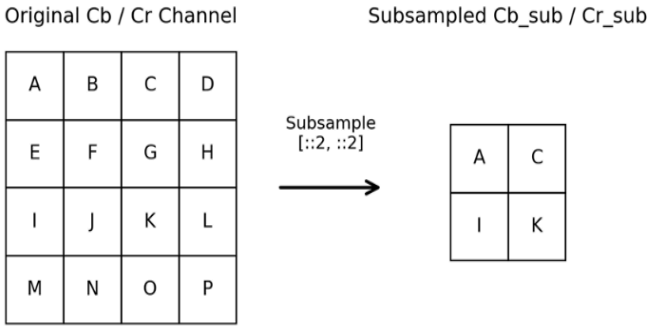
preprocess_frame(frame_bgr) combines color conversion and chroma subsampling. It returns a dictionary containing the original shape, the full Y channel, full chroma channels for visualization, and the subsampled chroma channels that are actually encoded.

```
def preprocess_frame(frame_bgr):
    Y, Cb, Cr = bgr_to_ycbcr(frame_bgr)
    Cb_sub, Cr_sub = subsample_420(Cb, Cr)
    return {
        "shape": frame_bgr.shape,
        "Y": Y,
        "Cb": Cb,
        "Cr": Cr,
        "Cb_sub": Cb_sub,
        "Cr_sub": Cr_sub,
    }
```

1.3 4:2:0 Chroma Subsampling

The code uses a simple 4:2:0 subsampling method. It keeps one chroma sample every two rows and two columns. This reduces the number of stored chroma samples to about one quarter of the original amount. For odd image dimensions, slicing produces approximately half the size, using ceil behavior for the sampled positions.

```
def subsample_420(Cb, Cr):
    Cb_sub = Cb[::2, ::2]
    Cr_sub = Cr[::2, ::2]
    return Cb_sub, Cr_sub
```



Important interpretation

Subsampling saves storage because Cb and Cr are smaller than Y. However, the discarded chroma values cannot be recovered exactly during reconstruction.

1.4 Reconstruction to BGR for Visualization

To display or save the reconstructed image, the function ycbcr_to_bgr(Y, Cb, Cr) upsamples the chroma channels back to the Y channel size. The code uses linear interpolation through cv2.resize. Then it stacks the channels in OpenCV YCrCb order and converts back to BGR.

```
def ycbcr_to_bgr(Y, Cb, Cr):
    h, w = Y.shape
    Cb_up = cv2.resize(Cb, (w, h), interpolation=cv2.INTER_LINEAR)
    Cr_up = cv2.resize(Cr, (w, h), interpolation=cv2.INTER_LINEAR)
    ycrcb = np.stack([Y, Cr_up, Cb_up], axis=2)
    ycrcb = np.clip(ycrcb, 0, 255).astype(np.uint8)
    return cv2.cvtColor(ycrcb, cv2.COLOR_YCrCb2BGR)
```

This operation restores the dimensions but not the exact original chroma information. Interpolation estimates missing values between known samples.

PART 2: Intra-frame Coding (I-frames)

An I-frame is encoded independently, without using any previous or future frame. This is similar to compressing a still image. The code applies 8x8 block DCT and quantization separately to the Y, Cb_sub and Cr_sub channels.

2.1 JPEG-like Quantization Matrices

The module defines two standard-style quantization matrices: one for luminance and one for chrominance. The luminance matrix is used for Y, while the chrominance matrix is used for Cb and Cr. Larger values cause stronger coefficient reduction, especially for high-frequency details.

CHROMA_Q	1	2	3	4	5	6	7	8
1	17	18	24	47	99	99	99	99
2	18	21	26	66	99	99	99	99
3	24	26	56	99	99	99	99	99
4	47	66	99	99	99	99	99	99
5	99	99	99	99	99	99	99	99
6	99	99	99	99	99	99	99	99
7	99	99	99	99	99	99	99	99
8	99	99	99	99	99	99	99	99

LUMA_Q	1	2	3	4	5	6	7	8
1	16	11	10	16	24	40	51	61
2	12	12	14	19	26	58	60	55
3	14	13	16	24	40	57	69	56
4	14	17	22	29	51	87	80	62
5	18	22	37	56	68	109	103	77
6	24	35	55	64	81	104	113	92
7	49	64	78	87	103	121	120	101
8	72	92	95	98	112	100	103	99

2.2 Quality Factor Scaling

The quality factor qf is a value from 1 to 100. The code clips invalid values into this range. qf = 50 keeps the base matrix almost unchanged. qf below 50 gives stronger compression and lower quality. qf above 50 gives weaker compression and higher quality.

```
if qf < 50:
    scale = 5000.0 / qf
else:
    scale = 200.0 - 2.0 * qf

q = floor((base_q * scale + 50) / 100)
q = clip(q, 1, 255)
```

2.3 Two-dimensional DCT and IDCT

The Discrete Cosine Transform changes a spatial pixel block into frequency coefficients. Low-frequency coefficients represent smooth areas and global intensity changes. High-frequency coefficients represent edges and fine details. The code applies scipy.fftpack.dct twice: first across one dimension, then across the other dimension.

```
def dct2(block):
    return dct(dct(block.T, norm="ortho").T, norm="ortho")

def idct2(block):
    return idct(idct(block.T, norm="ortho").T, norm="ortho")
```

2.4 Encoding One 8x8 Block

Before applying the DCT, the code subtracts 128 from pixel values. This level shift centers the values around zero, which is more suitable for DCT coding. Then coefficients are divided by the scaled quantization matrix and rounded to integers.

```
def encode_block(block, Q):
    shifted = block.astype(np.float32) - 128.0
    coeffs = dct2(shifted)
    quant      = np.round(coeffs /
    Q).astype(np.int16) return quant
```

2.5 Decoding One 8x8 Block

Decoding reverses the process approximately. Quantized coefficients are multiplied by Q, inverse DCT is applied, then 128 is added back. Because quantization rounded values during encoding, reconstruction is lossy.

```
def decode_block(quant, Q):
    coeffs = quant.astype(np.float32) * Q
    recon = idct2(coeffs) + 128.0
    return recon
```

Main source of loss

The DCT and IDCT are reversible mathematical transforms, but quantization is not fully reversible because coefficients are rounded.

2.6 Padding and Full-channel Coding

DCT coding requires blocks of 8x8. If the channel height or width is not a multiple of 8, the code pads the bottom and right borders using edge replication. The original shape is stored so that decoding can crop the channel back to its original size.

```
def pad_to_multiple(channel, block_size=8):
    h, w = channel.shape
    ph = (block_size - h % block_size) % block_size
    pw = (block_size - w % block_size) % block_size
    return np.pad(channel, ((0, ph), (0, pw)), mode="edge")
```

The function encode_channel(channel, Q) pads a channel, loops over every 8x8 block, encodes each block, and returns both the quantized blocks and original shape.

```
quant_blocks = np.zeros((bh, bw, 8, 8), dtype=np.int16)
for i in range(bh):
    for j in range(bw):
        block = padded[i*8:(i+1)*8, j*8:(j+1)*8]
        quant_blocks[i, j] = encode_block(block, Q)

return quant_blocks, original_shape
```

The function decode_channel reverses this by decoding every 8x8 block, placing the reconstructed blocks into a full image array, cropping to the original shape, and clipping values to the valid range [0, 255].

```
for i in range(bh):
    for j in range(bw):
        recon[i*8:(i+1)*8, j*8:(j+1)*8] = decode_block(quant_blocks[i, j],
            Q)

return np.clip(recon[:h, :w], 0, 255)
```

2.7 I-frame Encoding

The function encode_iframe(Y, Cb_sub, Cr_sub, qf=50) encodes a full I-frame. It first creates scaled quantization matrices, then encodes the luma channel with the luma matrix and the chroma channels with the chroma matrix.

```
def encode_iframe(Y, Cb_sub, Cr_sub, qf=50):
    Qy = scale_quantisation_matrix(LUMA_Q, qf)
    Qc = scale_quantisation_matrix(CHROMA_Q, qf)

    Y_blocks, Y_shape = encode_channel(Y, Qy)
    Cb_blocks, Cb_shape = encode_channel(Cb_sub, Qc)
    Cr_blocks, Cr_shape = encode_channel(Cr_sub, Qc)

    return {
        "type": "I",
        "qf": qf,
        "Y_blocks": Y_blocks, "Y_shape": Y_shape,
        "Cb_blocks": Cb_blocks, "Cb_shape": Cb_shape,
        "Cr_blocks": Cr_blocks, "Cr_shape": Cr_shape,
    }
```

Returned key	Meaning
type	Frame type. Here it is I.
qf	Quality factor used to scale the quantization matrices.
Y_blocks	Quantized 8x8 DCT blocks for the luma channel.
Y_shape	Original Y shape before padding.
Cb_blocks / Cr_blocks	Quantized 8x8 DCT blocks for subsampled chroma channels.
Cb_shape / Cr_shape	Original chroma shapes before padding.

2.8 I-frame Decoding

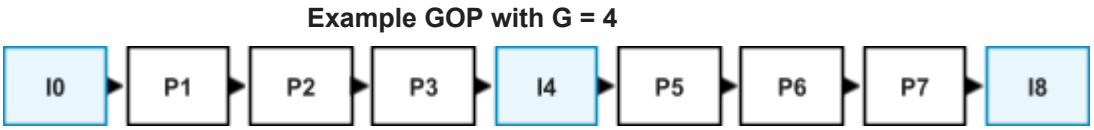
decode_iframe(frame_data) reads qf, rebuilds Qy and Qc, decodes Y, Cb_sub and Cr_sub independently, and returns the reconstructed channels. These channels can later be converted back to BGR for visualization.

```
def decode_iframe(frame_data):
    qf = frame_data["qf"]
    Qy = scale_quantisation_matrix(LUMA_Q, qf)
    Qc = scale_quantisation_matrix(CHROMA_Q, qf)

    Y          = decode_channel(frame_data["Y_blocks"],
        Qy, frame_data["Y_shape"])
    Cb_sub = decode_channel(frame_data["Cb_blocks"], Qc,
        frame_data["Cb_shape"])
    Cr_sub = decode_channel(frame_data["Cr_blocks"], Qc,
        frame_data["Cr_shape"])
    return Y, Cb_sub, Cr_sub
```

PART 3: Inter-frame Coding (P-frames)

A P-frame is not encoded independently. It is predicted from a previous reconstructed frame. The encoder stores motion vectors and residual information instead of storing the full frame directly. This reduces data when consecutive frames are similar.



Every G-th frame is encoded as an I-frame; intermediate frames are P-frames.

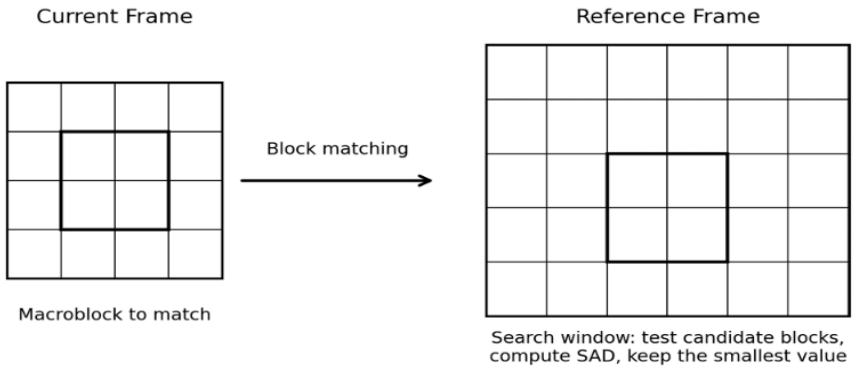
Figure 3 - GOP structure mentioned in the code comments.

3.1 Reference Frame Principle

The reference frame used by encode_pframe is the previous reconstructed frame, not the original previous frame. This is important because the decoder only has access to reconstructed frames. Using the reconstructed reference avoids mismatch between encoder and decoder.

3.2 Motion Estimation with Full Search

The function full_search_block_matching(current, reference, search_range=8) performs exhaustive block matching on the Y channel. The current and reference luma frames are padded to sizes that are multiples of 16. Each 16x16 macroblock in the current frame is matched against candidate blocks in the reference frame inside a search window of plus/minus search_range pixels.



3.3 SAD Criterion

For every candidate displacement (dy, dx), the code computes the sum of absolute differences. The displacement with the smallest SAD becomes the motion vector for that macroblock.

```
sad = np.sum(np.abs(cur_block - ref_block))
if sad < best_sad:
    best_sad = sad
    best_dy, best_dx = dy, dx
```

In formula form: SAD(dy, dx) = sum over the block of |current(y, x) - reference(y + dy, x + dx)|. A smaller SAD means the candidate block is more similar to the current block.

3.4 Motion Vector Array

The code stores motion vectors in an array of shape (mbh, mbw, 2). MBH is the number of macroblocks vertically and MBW is the number horizontally. The last dimension contains [dy, dx].

```
vertically,motion_vectors = np.zeros((mbh, mbw, 2), dtype=np.int32)
motion_vectors[i, j] = [best_dy, best_dx]
```

Element	Meaning
i, j	Macroblock position in the frame grid.
dy	Vertical displacement. Positive values mean the best block is lower in the reference frame.
dx	Horizontal displacement. Positive values mean the best block is to the right in the reference frame.
search_range	Maximum tested displacement in each direction. The default value is 8.

3.5 Predicted Frame Construction

Once the best vector is selected, the matching reference block is copied into the predicted frame. The reference is padded using edge values so that motion search can work near image borders.

```
ry = y0 + pad + best_dy
rx = x0 + pad + best_dx
predicted[y0:y0+MB_SIZE, x0:x0+MB_SIZE] = ref_padded[ry:ry+MB_SIZE,
    rx:rx+MB_SIZE]
```

3.6 Reapplying Motion Vectors

The function apply_motion_vectors(reference, motion_vectors, mb_size=16) reconstructs a predicted frame from a reference frame and a previously stored motion vector field. This is used during decoding, because the decoder does not repeat the motion search. It only applies the received vectors.

```
def apply_motion_vectors(reference, motion_vectors, mb_size=MB_SIZE):
    pad = max(int(np.abs(motion_vectors).max()) + 1, 1)
    ref_padded = np.pad(reference, pad, mode="edge")
    predicted = np.zeros_like(reference)

    for i in range(mbh):
        for j in range(mbw):
            y0, x0 = i * mb_size, j * mb_size
            dy, dx = motion_vectors[i, j]
            predicted[y0:y0+mb_size, x0:x0+mb_size] = ref_padded[
                y0+pad+dy:y0+pad+dy+mb_size,
                x0+pad+dx:x0+pad+dx+mb_size
            ]
    return predicted
```

3.7 Residual Computation

Motion prediction is usually not perfect. The difference between the current frame and the predicted frame is called the residual. If prediction is good, residual values are small and easier to compress.

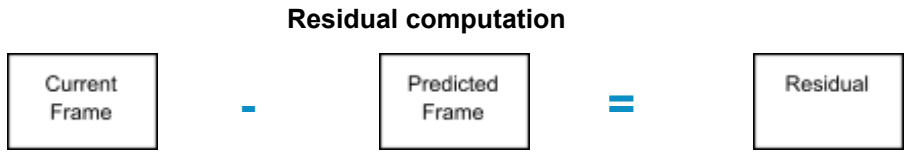


Figure 5 - The residual is the difference between the current frame and its prediction.

```
residual = current - predicted
```

Important difference from normal image blocks

Residual values are already centered around zero, so the residual encoder does not subtract 128 before DCT.

3.8 Residual Encoding and Decoding

The residual channels are compressed using 8x8 DCT blocks and quantization, similar to I-frame channels. The main difference is that no level shift is used.

```
        def encode_residual_channel(residual, Q):
            original_shape = residual.shape
            padded = pad_to_multiple(residual)
            quant_blocks = np.zeros((bh, bw, 8, 8), dtype=np.int16)

            for i in range(bh):
                for j in range(bw):
                    block = padded[i*8:(i+1)*8, j*8:(j+1)*8].astype(np.float32)
                    coeffs = dct2(block)       # no level shift
                    quant_blocks[i, j] = np.round(coeffs / Q).astype(np.int16)

            return quant_blocks, original_shape
```

Decoding multiplies quantized residual coefficients by Q, applies inverse DCT, and crops to the original residual shape. The result is added to the predicted frame.

```
        coeffs = quant_blocks[i, j].astype(np.float32) * Q
        recon[i*8:(i+1)*8, j*8:(j+1)*8] = idct2(coeffs)
        return recon[:h, :w]
```

3.9 Chroma Motion Vector Scaling

Motion vectors are estimated on the full-resolution Y channel. Since Cb_sub and Cr_sub are half-resolution channels, the motion vectors are divided by two before applying them to chroma.

```
        mv_cb = motion_vectors // 2
        pred_Cb = apply_motion_vectors(ref_Cb_pad, mv_cb, mb_size=MB_SIZE//2)
        pred_Cr = apply_motion_vectors(ref_Cr_pad, mv_cb, mb_size=MB_SIZE//2)
```

Example: If a Y motion vector is [4, -6], the corresponding chroma vector becomes [2, -3]. Because the code uses integer floor division, some precision is lost, especially for odd and negative motion values.

3.10 P-frame Encoding: Complete Procedure

encode_pframe(Y, Cb_sub, Cr_sub, ref_Y, ref_Cb_sub, ref_Cr_sub, qf=50, search_range=8) compresses one P-frame using the previous reconstructed frame as a reference.

Step	Operation in encode_pframe
1	Scale LUMA_Q and CHROMA_Q according to qf.
2	Pad Y and ref_Y so their height and width are multiples of MB_SIZE = 16.
3	Run full-search block matching on the Y channel.
4	Build predicted_Y_pad using the best reference blocks.
5	Compute residual_Y_pad = Y_pad - predicted_Y_pad.
6	Encode the Y residual with DCT and quantization.
7	Scale motion vectors by two for chroma because chroma is half resolution.
8	Pad Cb_sub and Cr_sub and their references to multiples of 8.
9	Predict Cb and Cr using the scaled vectors.
10	Compute chroma residuals and encode them with the chroma quantization matrix.
11	Return a dictionary containing frame type, qf, search range, motion vectors, shapes, and residual blocks.

```
        motion_vectors, predicted_Y_pad = full_search_block_matching(
            Y_pad, ref_Y_pad, search_range
        )

        residual_Y_pad = Y_pad - predicted_Y_pad
        residual_Y_blocks, residual_Y_shape =
            encode_residual_channel(residual_Y_pad, Qy)
```

3.11 P-frame Returned Data Structure

The P-frame dictionary contains everything needed by the decoder to reconstruct the frame from the reference. It does not store the full current frame directly.

Returned key	Purpose
type	Frame type. Here it is P.
qf	Quality factor used for both luma and chroma residual quantization.
search_range	Search range used during encoding, stored for information and reproducibility.
motion_vectors	Motion vector field of shape (mbh, mbw, 2).
Y_pad_shape	Shape of padded luma frame used for macroblock prediction.
Y_original_shape	Original luma shape before macroblock padding.
residual_Y_blocks	Quantized DCT blocks of the luma residual.
residual_Y_shape	Original residual Y shape before 8x8 DCT padding.
Cb_pad_shape	Shape of padded chroma frame used for chroma prediction.
Cb_original_shape	Original Cb_sub shape.
residual_Cb_blocks	Quantized DCT blocks of the Cb residual.
residual_Cb_shape	Original CB residual shape before 8x8 padding.
Cr_original_shape	Original Cr_sub shape.
residual_Cr_blocks	Quantized DCT blocks of the Cr residual.
residual_Cr_shape	Original Cr residual shape before 8x8 padding.

3.12 P-frame Decoding: Complete Procedure

decode_pframe(frame_data, ref_Y, ref_Cb_sub, ref_Cr_sub) reconstructs a P-frame. The decoder receives the P-frame dictionary and the same reference frame that the encoder used, but in reconstructed form.

Step	Operation in decode_pframe
1	Read QF and rebuild QY and QC.
2	Read the motion vectors from frame_data.
3	Pad the reference Y to Y_pad_shape.
4	Apply motion vectors to produce predicted_Y_pad.
5	Decode residual_Y_pad from residual_Y_blocks.
6	Compute recon_Y_pad = predicted_Y_pad + residual_Y_pad.
7	Crop back to Y_original_shape and clip values to [0, 255].
8	Scale motion vectors by two for chroma.
9	Pad reference Cb and Cr to Cb_pad_shape.
10	Predict Cb and Cr using scaled motion vectors.
11	Decode Cb and Cr residuals and add them to their predictions.
12	Crop chroma channels to their original shapes and clip to [0, 255].

```
        predicted_Y_pad = apply_motion_vectors(ref_Y_pad, mv, MB_SIZE)
        residual_Y_pad = decode_residual_channel(
            frame_data["residual_Y_blocks"], Qy, frame_data["residual_Y_shape"]
        )
        recon_Y_pad = predicted_Y_pad + residual_Y_pad
        Y = np.clip(recon_Y_pad[:h, :w], 0, 255)
```

4. End-to-End Encoder Pipeline

The complete encoder processes a sequence of frames. Each frame is first preprocessed. Then the GOP rule decides whether the frame is encoded as an I-frame or as a P-frame. Every G-th frame is an I-frame; other frames are P-frames predicted from the previous reconstructed frame.

```
for frame_index, frame_bgr in enumerate(video_frames):
    processed = preprocess_frame(frame_bgr)
    Y = processed["Y"]
    Cb_sub = processed["Cb_sub"]
    Cr_sub = processed["Cr_sub"]

    if frame_index % G == 0:
        encoded = encode_iframe(Y, Cb_sub, Cr_sub, qf)
        ref_Y, ref_Cb, ref_Cr = decode_iframe(encoded)
    else:
        encoded = encode_pframe(
            Y, Cb_sub, Cr_sub,
            ref_Y, ref_Cb, ref_Cr,
            qf=qf,
            search_range=search_range
        )
        ref_Y, ref_Cb, ref_Cr = decode_pframe(encoded, ref_Y, ref_Cb,
            ref_Cr)

    store(encoded)
```

Why the encoder decodes immediately

After encoding a frame, the encoder reconstructs it and uses that reconstructed frame as the next reference. This keeps encoder and decoder synchronized.

4.1 End-to-End Decoder Pipeline

The decoder reads the stored encoded frames in order. I-frames are decoded independently. P-frames are decoded using the previous reconstructed frame and the stored motion vectors/residuals.

Simplified Decoding Pipeline for an I-frame

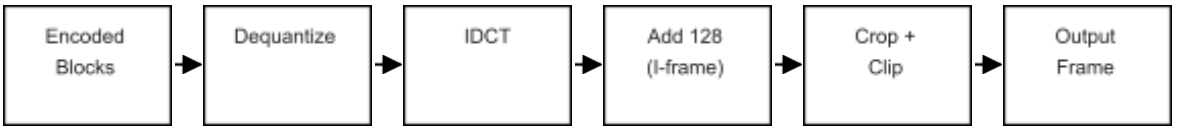


Figure 6 - Simplified I-frame decoding path. P-frame decoding adds prediction from motion vectors.

4.2 Quality Evaluation

The report can evaluate quality by comparing original and reconstructed images or frames. The code comments mention maximum difference and mean difference. These values summarize how much reconstruction differs from the input.

Metric	Meaning	Interpretation
Maximum absolute difference	Largest pixel error observed in any channel.	A high value may indicate strong distortion in a small region or a few pixels.
Mean absolute difference	Average error over all pixels.	A high value indicates stronger global degradation.
Visual inspection	Human evaluation of reconstructed frames.	Important because numerical error does not always match perceived quality.
Bitrate or size	Amount of data used to store the encoded frames.	Not fully implemented because there is no entropy coding or final bitstream format.

4.3 Important Implementation Details

Topic	Implementation detail
Data types	Input BGR is uint8. Y, Cb and Cr become float32. Quantized blocks are int16. Motion vectors are int32.
Clipping	Decoded image channels are clipped to [0, 255] to remain valid pixel values.
Padding mode	The code uses mode="edge," which repeats border pixels. This avoids introducing black borders.
Block sizes	DCT uses 8x8 blocks. Motion estimation uses 16x16 macroblocks for luma and 8x8 blocks for subsampled chroma prediction.
Reference consistency	P-frames must reference previously reconstructed frames, not original frames.

5. Function-by-function Summary

Function	Input	Output	Role
bgr_to_ycbcr	BGR frame H x W x 3	Y, Cb, Cr	Converts OpenCV BGR to YCbCr components.
ycbcr_to_bgr	Y, Cb, Cr	BGR frame	Upsamples chroma if needed and converts back to displayable BGR.
subsample_420	Cb, Cr	Cb_sub, Cr_sub	Reduces chroma resolution using [::2, ::2].
preprocess_frame	BGR frame	dictionary	Groups all preprocessing outputs.
scale_quantization_matrix	base matrix, qf	scaled matrix	Controls compression strength.
dct2 / idct2	8x8 block	transformed block	Forward and inverse frequency transform.
encode_block	8x8 block, Q	int16 coefficients	Level shift, DCT, and quantization.
decode_block	quantized block, Q	reconstructed block	Dequantization, IDCT, and add 128.
pad_to_multiple	2D channel	padded channel	Makes dimensions divisible by block size.
encode_channel	full channel, Q	blocks and shape	Encodes all 8x8 blocks in one channel.
decode_channel	blocks, Q, shape	channel	Reconstructs a full channel and crops it.

5. Function-by-function Summary (continued)

Function	Input	Output	Role
encode_iframe	Y, Cb_sub, Cr_sub, qf	I-frame dictionary	Encodes a complete I-frame.
decode_iframe	I-frame dictionary	Y, Cb_sub, Cr_sub	Reconstructs an I-frame.
full_search_block_matching	current Y, reference Y, search range	motion vectors, predicted Y	Finds best 16x16 block matches using SAD.
apply_motion_vectors	reference, motion vectors, block size	predicted frame	Builds prediction from stored vectors.
encode_residual_channel	residual, Q	blocks and shape	Encodes prediction error using DCT and quantization.
decode_residual_channel	residual blocks, Q, shape	residual	Reconstructs residual without adding 128.
encode_pframe	current channels + reference channels	P-frame dictionary	Encodes motion vectors and residual blocks.
decode_pframe	P-frame dictionary + reference channels	Y, Cb_sub, Cr_sub	Reconstructs a P-frame from prediction and residuals.

Part 4: Lossless Entropy Coding and Bitstream Serialization

3.13 Structured Object Graph Aggregation

To achieve a fully operational video codec pipeline capable of compiling compressed data into a unified, compliance-ready file format, the disjoint structural outputs from the spatial and temporal encoding loops must be aggregated. For every sequential frame processed, the encoding engine automatically groups all volatile variables—specifically macroblock-level coordinate motion vector tuples, block-partitioned quantized DCT matrices (8 *8), frame-type index strings, and tracking spatial shapes—into a single structured Python native mapping instance.

3.14 High-Protocol Native Serialization

To compile this multidimensional frame mapping dictionary into a contiguous byte sequence, the architecture avoids slow, manual bit-shifting layouts or error-prone custom byte alignment configurations. Instead, it interfaces directly with the native serialization framework via pickle.dumps(), forcing the implementation parameters to use pickle.HIGHEST_PROTOCOL. This engineering choice abstracts data boundaries natively, preserving the complex nested array architectures of the video frames at maximum processing speed.

3.15 Maximum-Density Deflate Compaction

To strictly minimize the final storage footprint of the generated bitstream on disk, the raw serialized byte stream is passed into a high-density lossless compression loop. The pipeline feeds the object stream into an optimized compaction routine powered by the zlib library, configured to its absolute maximum execution ceiling (level=9). This forces the underlying compression engine to perform deep dictionary sliding-window searches (LZ77) and generate highly optimized Huffman tree structures. This process is remarkably effective at flattening the extensive runs of zero-valued coefficients generated by the lossy quantization matrices.

3.16 Binary Framing and Fixed-Length Header Prefixes

The resulting compressed bitstream payload is written to a structural binary output file (.bin). To guarantee secure parsing during decoding, the file architecture implements a custom binary framing layout:

The Framing Boundary Layer: An uncompressed 4-byte big-endian unsigned integer prefix (>I) is generated at the absolute front of the file, explicitly encoding the exact byte length of the upcoming metadata block.

The Validation Metadata Block: This section encapsulates unique magic verification bytes (b"MMPEG4") to confirm file integrity, followed by vital operational configuration fields (such as total frame count, height, and width tracking values).

3.17 Symmetric Bitstream Decoding Pass

The inverse decoding routine unrolls this custom architecture sequentially. The playback engine reads the initial 4-byte boundary layer, pulls and parses the uncompressed validation metadata block to calibrate the target reconstruction matrix shapes, and decompresses the remaining payload via zlib.decompress(). Finally, pickle.loads() deserializes the recovered stream, restoring the operational dictionaries to feed the reconstruction loops for the I-frame and P-frame macroblocks.

Part 5 - Quality Evaluation Framework and Dashboard Visualization

5.1 Rate Calculation and Memory Footprint Analytics

The evaluation layer systematically monitors the data footprint reduction achieved by the codec. It automatically tracks the total uncompressed raw visual volume, defined mathematically as:

Raw Footprin = H * W * 3 bytes (per BGR frame array)

This theoretical memory size is cross-referenced directly against the exact physical storage footprint of the generated .bin file on disk to output the precise operational compression ratio:

$$\text{Compression Ratio} = \frac{\text{Total Uncompressed Size(Bytes)}}{\text{Compressed File Size (Bytes)}}$$

5.2 Spatial Distortion Assessment (MSE & PSNR)

To strictly measure the mathematical degradation introduced by the lossy sections of the pipeline (Quantization), the system runs a frame-by-frame Peak Signal-to-Noise Ratio (PSNR) analysis calculated in the double-precision floating-point domain. For an original frame I_{orig} and its reconstructed version I_{recon}, the Mean Squared Error (MSE) across the three color channels is calculated as:

$$\text{MSE}= \frac{1}{3 \cdot H \cdot W} \sum_{c=1}^{3} \sum_{y=1}^{H} \sum_{x=1}^{W} (I_{orig}(x,y,c) - I_{recon}(x,y,c))^2$$

The absolute Peak Signal-to-Noise Ratio formula is thus defined as:

$$\text{PSNR}=10.log_{10}(\frac{255^2}{MSE})$$

To safely handle perfect matching states where $MSE = 0$, a protective logical boundary intercepts the math routine and forces the output to an infinity float representation (float("inf")), preventing fatal division-by-zero errors.

5.3 Headless Dashboard Matrix Rendering

To visualize the state of the codec end-to-end, a comprehensive pipeline dashboard is rendered using matplotlib.gridspec under a headless server configuration (matplotlib.use("Agg")). This headless configuration uncouples the visualization module from system display drivers or active X11 environments, ensuring safe execution on terminal-only cloud servers. The interface maps five parallel rows:

Input Sequence: Shows the original sequential BGR source frames.

Color Space Processing: Displays individual isolated Y, Cb, and Cr color channels.

Transform Walkthrough: Tracks an individual 8 *8 pixel block across every stage (Raw Pixels □DCT Coefficients □Quantized Levels □Reconstructed Block).

Motion Field Overlay: Renders spatial displacement fields by overlaying motion vector arrows onto predicted P-frames.

Residual Maps & Analytics: Displays a frame-by-frame absolute error map (Absolute Difference * 5), alongside an analytical bar chart tracking PSNR variations and a pie chart breaking down the frame-type distribution.

6. Core Technical Justifications and Design Choices

- Native Object Serialization via High-Protocol Abstraction

Choice: Object graph serialization using pickle set to protocol=pickle.HIGHEST_PROTOCOL.

Justification: Writing low-level parsing scripts or manual bit-packing layers introduces significant complexity and potential data alignment errors when handling heterogeneous data streams (such as mixing matrix blocks, tuple coordinates, and tracking frames). Utilizing high-protocol native serialization abstracts data boundaries seamlessly. Given that this deployment is designed as a robust proof-of-concept pipeline, this approach ensures absolute structural integrity, maximizes development speed, and allows any minor structural layout overhead to be compacted efficiently during the subsequent zlib deflate stage.

- Computational Priority of High-Density Deflation

Choice: Maximum-density stream compression using zlib.compress with level=9.

Justification: Production video implementations prioritize maximizing storage savings on disk over minor adjustments in CPU compute time during the encoding pass. Forcing zlib to level 9 configures the deflation core to perform deeper sliding-window dictionary searches. This choice is ideal for capturing and compressing the long sequences of zero-valued high-frequency coefficients created during quantization, maximizing overall storage efficiency.

- Header Decoupling via Fixed-Length Prefixing

Choice: Fixed 4-byte big-endian byte-length length prefixing ahead of the magic validation header.

Justification: Injecting an uncompressed, highly deterministic 4-byte boundary layer at the head of the file implements an essential safety check for the decoder loop. This enables the playback engine to immediately parse and inspect global video attributes (such as resolution shapes, frame counts, and profile configurations) without executing an expensive full-file bitstream decompression first.

7. Experimental Analysis and Performance Sweeps

The automated testing routines embedded within the evaluation layer (plot_compression_vs_qf and plot_compression_vs_gop) allow for systematic profiling of the pipeline across varying configurations.

- Rate-Distortion Behavior as a Function of Quality Factor (QF)

Compression Behavior: Testing across a broad sweep of Quality Factors shows that as QF values decrease (e.g., QF = 10), the quantization step sizes scale up dramatically. This forces the higher-frequency DCT coefficients to drop to zero. The lossless entropy compaction stage (zlib) exploits this uniformity, resulting in high compression ratios. Conversely, as the QF approaches 95, the quantization step size shrinks, preserving fine structural details and high-frequency content. This increases bitstream complexity and results in a larger final .bin file size.

Reconstruction Quality Profile: The measured peak signal-to-noise ratio follows a clear logarithmic curve. Low QF parameters introduce visible blocking artifacts, reducing the mean PSNR. Elevating the QF recovers fine spatial details and object boundaries, moving the mean PSNR into an optimal rate-distortion balance zone, typically observed between QF=50 and QF=70.

- Inter-Frame Coding Efficiency as a Function of GOP Size

Compression Efficiency Gains: Running sweeps with a Group of Pictures size of GOP = 1 (All-I-frame mode) forces each video frame to compress independently. This approach relies solely on spatial compression and completely ignores temporal redundancies, yielding the lowest compression efficiency. As the GOP size expands, more P-frames are introduced into the stream. Because P-frames only need to store compact motion vectors and sparse high-frequency residual matrices, the entropy engine compresses these structures highly efficiently. This causes the overall compression ratio to rise rapidly before stabilizing at larger GOP sizes.

Temporal Quality Constraints and Distortion Drift: While extending the GOP size significantly reduces file size, it introduces temporal distortion propagation. Because P-frames are predicted from previously reconstructed, lossy reference frames, small quantization errors compound frame-by-frame.

This leads to a gradual quality drift, demonstrating that periodic I-frame insertions are essential to clear out accumulated motion-compensation drift and reset the baseline quality anchor.

Conclusion:

This practical project allowed us to design and validate a complete, functional video compression pipeline inspired by the principles of the MPEG-4 standard. By combining color processing (Part 1), spatial coding via DCT and quantization for Intra-frames (Part 2), and temporal coding via motion estimation for Predicted frames (Part 3), the system effectively eliminates visual redundancies. The finalization of the pipeline highlighted the importance of entropy coding using pickle and zlib (Part 4) to significantly reduce the final storage footprint on disk. Finally, the evaluation module (Part 5) validated the essential trade-offs between visual quality (PSNR) and compression ratio across varying Quality Factors and GOP sizes. This teamwork provides a concrete, comprehensive understanding of the foundational building blocks that make up modern video codecs.